

第1回：LLMイントロダクション

LLMアプリケーション開発 2026

外部の資料を使って答えるアプリを作る

今日のゴール

- LLMが文章を生成する仕組みを、自分の言葉で説明する
- モデル、チャットサービス、APIの違いを説明する
- 資料を渡す理由と、回答を確かめる必要性を理解する

今日はコードの実装を求めません。完成品を観察します。

13回の授業

回	内容
1～3	LLMの基礎、API、Webアプリとの連携
4～6	外部資料、キーワード検索、意味検索
7～8	関数の呼び出し、評価、安全な運用
9～13	自分の題材で企画・制作・改善・発表

毎回、同じアプリの機能を増やします。

前の授業とのつながり

既習の技術	今回の使い道
HTML・CSS・JavaScript	質問フォームと回答の画面
FastAPI・JSON・fetch	ブラウザとサーバーの通信
SQLite・SQL	参照資料と架空の課題の保存
入力検証・textContent	不正な入力や出力への対策

新しく加えるのは、外部のLLM APIと知識の検索です。

LLMとは

Large Language Model：大規模言語モデル。

大量の文章などから言葉の関係を学習し、入力された文脈に続くトークンを予測して文章を生成します。

質問への回答、要約、分類、文章の書き換えなどに利用できます。
自然な文章であることと、内容が正しいことは別に確認します。

言語モデルの簡単な歴史

時期	主な考え方
従来	直前の単語の並びから、次の単語の頻度を推定
2000～2010年代	ニューラルネットワークで言葉の特徴を学習
2017年	Transformerを提案した論文が発表
2018年以降	大規模な事前学習を使うモデルが発展
2022年以降	対話形式のサービスが普及

Transformerの考え方

「私は銀行へ行き、お金を預けた」

どの言葉に注目すると、「銀行」の意味を理解しやすいでしょうか。
Attentionは、文脈内の要素どうしの関係を計算する仕組みです。

この授業では数式や学習処理の実装には立ち入りません。
入力にどんな文脈を与えるかに注目します。

トークンと生成

トークンは、モデルが扱う文章の区切りです。
1文字や1単語と必ず一致するわけではありません。

1. 入力をトークンに分ける
2. 文脈から次のトークンの候補を計算する
3. 選んだトークンを文脈に加える
4. 終了条件まで繰り返す

「文章の続きを作る」仕組みを、質問への応答にも使います。

学習と推論

処理	何をするか	この授業
学習	データからモデルのパラメータを調整する	実施しない
推論	既に学習したモデルに入力を渡し、出力を得る	APIで利用

授業資料を質問に添付するだけでは、モデルを再学習しません。
今回の回答で参照する情報として渡します。

モデル・サービス・API

言葉	役割	例
モデル	入力から出力を生成する仕組み	GPT、Claude、Geminiなど
サービス	人が画面から利用する製品	チャット画面など
API	プログラムから利用する窓口	自作アプリからの呼び出し

画面にある検索・履歴・ファイル機能を、モデル自体の能力と混同しないことが大切です。

代表的なモデル系列

系列	提供元	観察する点
GPT	OpenAI	APIの機能、用途ごとのモデル
Claude	Anthropic	APIの機能、用途ごとのモデル
Gemini	Google	入出力の種類、APIの機能
Llama	Meta	公開された重み、利用条件、実行環境

世代、利用条件、対応機能は変わります。公式資料で確認します。
順位を暗記するのではなく、用途に合わせて比べます。

この授業で使うモデル

文章生成と関数選択には **gpt-6-luna** を使います。

第2回から Responses API で呼び出します。

第6回の文章のベクトル化には、別用途の

text-embedding-3-small を使います。

生成用モデルと Embedding 用モデルは、仕事が異なります。

得意なことと、確認が必要なこと

使い方	確認すること
長い文の要約	大切な条件を落としていないか
問い合わせの分類	あいまいな例をどう扱うか
コードの説明	実際のコードと一致しているか
授業案内の回答	資料に根拠があるか

もっともらしい誤答を、ハルシネーションと呼ぶことがあります。

モデルが知らない情報

「このクラスの次の提出物は何ですか？」

モデルが一般的なプログラミング知識を持っていても、このクラスの案内を知っているとは限りません。

- 資料を渡す
- 必要な情報をDBから取得する
- 根拠がないときは確認できないと伝える

このための仕組みをWebアプリ側に作ります。

サンプルアプリの体験

リポジトリのルートから実行します。

```
source .venv/bin/activate  
cd session01/exercise  
python main.py
```

Codespacesの「ポート」から8000番をブラウザで開きます。
終了はターミナルで Ctrl+C です。

デモモードの意味

初期設定では外部APIを呼びません。

方式	デモでの動作
資料なし	決まった文章を表示
資料全体・検索	選んだ資料の本文を表示
課題DB	固定で未提出の架空データを取得

これは生成精度の実験ではありません。

本物の生成の比較は、講師のAPIモードの実演で観察します。

実習1：資料の有無

1. 「資料なし」で「第7回の提出物は？」と質問
2. 「資料全体を渡す」で同じ質問
3. 画面の資料名、見出し、本文を確認
4. 回答と、アプリが参照した情報の違いを記録

`exercise/worksheet.md` に観察結果を書きます。

授業案内はサンプル専用の架空データです。

実習2：資料にない質問

「来年度の試験日は？」と質問します。

- 参照資料に日付が書かれているか
- どのように答えると利用者が誤解しないか
- 日付を創作して回答すると、何が困るか

デモの抜粋表示と、講師の実際の生成結果を分けて記録します。

アプリの役割分担

場所	仕事
ブラウザ	質問と方式を送信し、結果を表示
FastAPI	入力を確認し、資料や関数を選ぶ
SQLite	資料や課題を保存
LLM API	渡された内容から回答や関数の引数を生成

LLMへ渡す情報と、実行できる操作をアプリ側が決めます。

確認問題

1. 資料を質問に添付すると、モデルを再学習する？
2. 回答に資料IDが付いていれば、正しいと判断できる？
3. デモの固定文だけで、モデルの性能を評価できる？
4. 生成モデルとEmbeddingモデルの役割は同じ？

理由を添えて、隣の人に説明してください。

提出物と次回

提出： `exercise/worksheet.md` のGitHub URL

- モデル、サービス、APIの違い
- 資料の有無で変わる事
- 資料にない質問への望ましい応答

次回はPythonから **gpt-6-luna** のAPIを呼び出します。